Kick Start 2021 - Round H

Analysis: Dependent Events

For this problem, we are given a graphical model in the form of a directed, rooted tree, where each vertex is an event. For any vertex v on this tree, let $v = 0$ and $v = 1$ denote the non-occurrence and occurrence of event v, respectively. Let l_j be the lowest common ancestor (LCA) of $\mathbf{u_j}$ and $\mathbf{v_j}$. By the total probability rule, $P[\mathbf{u_j} = 1, \mathbf{v_j} = 1] = \sum_{i=0}^{1} P[\mathbf{u_j} = 1, \mathbf{v_j} = 1, l_j = i]$. We know $\mathbf{u_j}$ and $\mathbf{v_j}$ are conditionally independent given l_j, because the paths $l_j \to \mathbf{u_j}$ and $l_j \to \mathbf{v_j}$ are edge-disjoint due to the definition of LCA. This allows us to simplify the earlier sum into $\sum_{i=0}^{1} P[\mathbf{u_j} = 1 | l_j = i] P[\mathbf{v_j} = 1 | l_j = i] P[l_j = i]$. This formula forms the basis of our solution, and the approaches for the two test sets differ only in how to compute the required probabilities efficiently.

To deal with the output format of this problem, we store all results as fractions and take the numerator and denominator modulo $10^9 + 7$ after each operation to avoid overflow. If our final result is $\frac{p}{q}$, we output $pq^{-1} \pmod{10^9 + 7}$. Due to the nature of the problem, q will always be a power of ten, so the inverse is guaranteed to exist, and can be efficiently computed via extended Euclidean algorithm. Alternatively, due to Fermat's little theorem, we can equivalently raise q to the $(10^9 + 5)$-th power to find its inverse; this can be done efficiently with exponentiation by squaring.

Test set 1

For this test set it suffices to naively compute l_j. We can then compute $P[\mathbf{u_j} = 1 | l_j]$ and $P[\mathbf{v_j} = 1 | l_j]$ by walking back down the tree from l_j. To do this, we can use the total probability rule similar to before; if $p(x)$ denotes the parent of x, then $P[x = 1 | l_j = i] = \sum_{k=0}^{1} P[x = 1, p(x) = k | l_j = i] = \sum_{k=0}^{1} P[x = 1 | p(x) = k] P[p(x) = k | l_j = i]$; note that the first term in this product comes from the input, and the second term comes from the previous step of our walk. We can use the same formula to compute $P[l_j = i]$, since $P[l_j = i] = \sum_{k=0}^{1} P[l_j = i | r = k] P[r = k]$, where r is the root of the tree.

Each vertex is visited at most $O(1)$ times per query using this technique, so our overall time complexity is $O(\mathbf{NQ})$ per test case, which is sufficient.

Test set 2

There are both more queries and a larger tree in this test set, so the naive solution is too slow. This test set requires computing l_j, the LCA of $\mathbf{u_j}$ and $\mathbf{v_j}$, in $O(\log \mathbf{N})$ time anyways, so we can think of ways to augment an LCA algorithm to also keep track of the necessary probabilities for us. For example, the binary lifting LCA algorithm pre-computes $p^i(v)$ for all vertices v, where $p^i(\cdot)$ gives the 2^i-th parent of a given vertex. We can extend the binary lifting algorithm to not only store the id of this parent, but to also store $P[v = 1 | p^i(v)]$. These probabilities can then be multiplied such that only $O(\log \mathbf{N})$ hops are needed to get from $\mathbf{u_j}$ or $\mathbf{v_j}$ up to l_j. The unconditional probability $P[l_j]$ can simply be pre-computed for the entire tree via DFS in $O(\mathbf{N})$ time. This allows us to answer each query in $O(\log \mathbf{N})$ time, so our overall time complexity is $O(\mathbf{N} \log \mathbf{N} + \mathbf{Q} \log \mathbf{N})$ per test case.

Kick Start 2021 - Round H

Analysis: Painter

Given the three primary colors: *Red*, *Yellow*, and *Blue*, we need to determine the minimum number of strokes required to paint a 1-dimensional painting $\mathbf{P}$ of length $\mathbf{N}$. In 1 stroke, you can choose a color either *Red*, *Yellow*, or *Blue*, and two integers l and r, such that $1 \leq l \leq r \leq \mathbf{N}$, and apply the chosen color to all squares $\mathbf{P_j}$ such that $l \leq j \leq r$.

Note as the colors: *Red*, *Yellow*, and *Blue* are independent of each other and the proportion and order of each color in the combination does not matter, the minimum number of strokes to complete the paining will be the minimum number of *Red* strokes $+$ minimum number of *Blue* strokes $+$ minimum number of *Yellow* strokes.

Let $F(x)$ be the minimum number of strokes needed of primary color x. Notice minimum number of strokes of x is same as number of continuous blocks of squares that will need the primary color x to form the required color. To find $F(x)$ we will start from the leftmost end of the squares. Whenever we encounter a square either with the color x or with some color y which is a combination of x and other primary colors, we paint the squares with the color x in 1 stroke until we reach the rightmost end, or some color y which is neither x nor a combination of x and any other primary color. We continue this until we have encountered all the squares. The number of strokes used is equal to $F(x)$.

Test Set 1

We are given that $\mathbf{P_i}$ will be one of $\{Y, B, G\}$. The final answer will be $F(Yellow) + F(Blue)$.

Complexity : $O(\mathbf{N})$ *per test case*

Test Set 2

We are given that $\mathbf{P_i}$ will be one of $\{U, R, Y, B, O, P, G, A\}$. The final answer will be $F(Red) + F(Yellow) + F(Blue)$.

Complexity : $O(\mathbf{N})$ *per test case*

Sample Code(C++)

```
int minimum_number_of_strokes_for_primary_color_x(string S, int N, char x, map<char, string> color_to_primar
  int min_strokes = 0;
  int last_stroke_position = INT_MIN;
  for(int i = 0; i < N; i++) {
    if (S[i] == 'U') {
      continue;
    }
    if(S[i] == x || color_to_primary_colors[S[i]].find(x) != string::npos) {
      if (last_stroke_position != i - 1) {
        min_strokes++;
      }
      last_stroke_position = i;
    }
  }
  return min_strokes;
}
```

Kick Start 2021 - Round H

Analysis: Silly Substitutions

In this problem, each replacement shortens the string by one character, so the number of replacements is no more than $\mathbf{N}$. It is the complexity of each replacement that may bother us.

Test Set 1

The basic step that we are asked to do is the "replace all" operation; that is, to find all the occurrences of a given substring within a given string, and replace them with another string.

Many programming languages have a built-in function that does exactly this; for example, in both Python and Java it is called `replace()`. In other languages you may implement the "replace all" operation by using a few basic operations or completely from scratch. Either way, the typical implementation of "replace all" would take *linear* time, because it has to:

- search the entire string for all occurrences of the substring, and
- (if found) build the resulting string.

And then, we are asked to apply "replace all" until there is nothing to change. This would look like (pseudo-code):

```
while True:
  previous = s
  s = s.replace_all('01', '2').replace_all('12', '3')....replace_all('90', '1')
  if s == previous:
    return s
```

This loop makes up to $\mathbf{N}$ iterations, each iteration attempts the "replace all" operation 10 times, and, as said, each attempt takes $O(\mathbf{N})$.

Therefore, *complexity:* $O(\mathbf{N}^2)$ *per test case.*

Test Set 2

To reduce the complexity we should implement our own "replace all" and optimize the two things that it does:

- Instead of searching the entire string every time, we will maintain a set of "locations of interest". More accurately, 10 sets: all locations of the substring `01`, all locations of the substring `12`, and so on. How to define a location in a string that keeps changing? The next bullet solves this.
- To quickly replace characters, we will convert the string to a bidirectional linked list where each node holds a character. Thus, a location in the string is actually a pointer to a node, and each local change, such as removing two nodes or inserting a node, takes $O(1)$ time.

A "replace all" operation might still take linear time in the optimized version. For example, if $\mathbf{S} =$ `010101010101` then the first replacement (of all the `01`s to `2`s) will have to replace $\mathbf{N}/2$ occurrences. But then, the next operations will be either few enough or fast enough to compensate for the long one. This is known as amortized analysis.

More accurately, whenever a location becomes interesting, one of the future replacement operations will have to handle it (as said, by $O(1)$ manipulations of the linked list). How many times can locations become interesting? In the initial string, maybe all the $\mathbf{N}$ characters (except the last one) start as interesting. Later on, whenever we replace a pair of characters with a new one, that new character may turn one or both its neighbors to be interesting. For example, when changing `1013` to `123` it creates two interesting substrings: `12` and `23`. Finer analysis can show that we only need to mark the `12` as interesting, but it does not really matter for our conclusion.

So, since we replace characters at most $\mathbf{N}$ times, the event of a location becoming interesting can happen at most $3\mathbf{N}$ times (or $2\mathbf{N}$ with finer analysis).

Therefore, *complexity:* $O(\mathbf{N})$ *per test case*

Notes:

1. We can even mark "false interesting locations". That is, mark *all* the initial locations and *both* neighbors of every modification as interesting, even if they do not really start a pair that should be replaced (like `01`). Of course, our "replace all" operation will need to check if each "interesting location" should indeed be replaced, but this is a good practice anyway. The best part is that our analysis with the $3\mathbf{N}$ bound will still be valid.
2. The *set* of locations of interest is indeed a set - each location should be marked once and the order that we handle them does not matter. However, if your programming language makes it harder (higher complexity) to maintain a set, you may as well store the locations of interest in any other structure, like a vector or a list.

Kick Start 2021 - Round H

Analysis: Transform the String

Let us first define the two operations that we can perform.

- **Clockwise**: Changing a letter to the one following it. For example, changing from `c` to `d`.
- **Counter-clockwise**: Changing a letter to the one preceding it. For example, changing from `a` to `z`.

Let us denote the ASCII value of a character c_x by $ASCII(c_x)$. If we move the padlock from a character c_a to another character c_b such that $ASCII(c_a) < ASCII(c_b)$, the number of operations required in clockwise direction = $ASCII(c_b) - ASCII(c_a)$ and the number of operations required in counter-clockwise direction = $26 - (ASCII(c_b) - ASCII(c_a))$.

For example, if we move the padlock from `c` to `e`:

- Number of operations required in clockwise direction = $ASCII(e) - ASCII(c) = 2$.
- Number of operations required in counter-clockwise direction = $26 - (ASCII(e) - ASCII(c)) = 24$.

Similarly, if we move the padlock from a character c_a to another character c_b such that $ASCII(c_a) > ASCII(c_b)$, the number of operations required in clockwise direction = $26 - (ASCII(c_a) - ASCII(c_b))$ and the number of operations required in counter-clockwise direction = $ASCII(c_a) - ASCII(c_b))$.

For example, if we move the padlock from `g` to `b`:

- Number of operations required in clockwise direction = $26 - (ASCII(g) - ASCII(b)) = 21$.
- Number of operations required in counter-clockwise direction = $ASCII(g) - ASCII(b) = 5$.

Thus minimum number of operations required to change a character in the padlock from c_a to c_b = $min(abs(ASCII(c_a) - ASCII(c_b)), 26 - abs(ASCII(c_a) - ASCII(c_b)))$.

Let us call the above expression $f(c_a, c_b)$.

Approach 1

Test Set 1

When length of $\mathbf{F}$ = 1 we need to change every character in $\mathbf{S}$ to that in $\mathbf{F}$. Therefore, the answer is the sum of $f(c_s, c_f)$ for every character c_s in $\mathbf{S}$ and c_f in $\mathbf{F}$.

Test Set 2

For this case, $\mathbf{F}$ can have multiple characters.

For each character in $\mathbf{S}$ we need to find a character in $\mathbf{F}$ such that $f(c_s, c_f)$ is minimized. Therefore, for every character c_s in $\mathbf{S}$, we iterate over all possible characters c_f in $\mathbf{F}$ and find minimum of $f(c_s, c_f)$ and add the minimum value to the final answer.

Approach 2

For each character in $\mathbf{S}$, move the padlock in the clockwise direction and count the number of operations until we reach a character that belongs to $\mathbf{F}$. Similarly, for the same character in $\mathbf{S}$, move the padlock in the counter-clockwise direction and count the number of operations until we reach a character that belongs to $\mathbf{F}$. Compare the number of operations in both directions and add minimum of the two to the final answer.